

PROJECT REPORT

Cafe Billing System

A command-line billing program in Python

Submitted by: Tapish Khantwal

Registration No.:26BCE11104

B.Tech Computer Science and Engineering (First Year)

VIT Bhopal University

Faculty: T.Venkateswara Rao

Course: Vityarthi Project

Date: 29 September 2026

1. Introduction

Billing in a small cafe is often done on paper or with a calculator. This works for a few orders but becomes slow and error-prone. In this project I built a simple billing system in Python that runs in the terminal.

The program shows a menu, lets the user add and remove items, and prints a final bill that includes 5% GST. I used only basic Python concepts that we have covered in class: dictionaries, functions, loops, conditions and string methods.

2. Problem Statement

Manual billing in cafes is slow and mistakes are common. The problem is to create a program that takes a customer's order, lets them change it before finishing, and calculates the final bill (with tax) correctly.

3. Functional Requirements

- The system should display a menu of items with their prices.
- The user should be able to add an item to the cart by typing its name.
- The user should enter a quantity for each item; adding the same item again increases its quantity.
- The user should be able to remove an item or reduce its quantity.
- The user should be able to view the cart and the menu at any time.
- The system should generate a bill with subtotal, 5% GST and total.
- The system should not generate a bill if the cart is empty.

4. Non-functional Requirements

- Usability- simple text commands, and clear messages for wrong input.
- Reliability- invalid quantities (letters, zero, negative) and unknown items must not crash the program.
- Performance- responds instantly since all data is kept in memory.
- Portability- runs on any system with Python 3, no extra libraries.
- Maintainability- each task is a separate function so the code is easy to read and change.

5. System Architecture

The program is a single Python file with three parts:

- Data: a menu dictionary (item name to price), a cart dictionary (item name to quantity).
- Functions: `show_menu`, `show_cart`, `add_item`, `remove_item` and `print_bill`.
- Main loop: a while loop that reads what the user types and calls the right function until the user types "done" with a non-empty cart.

6. Design Diagrams

a. Use Case Diagram

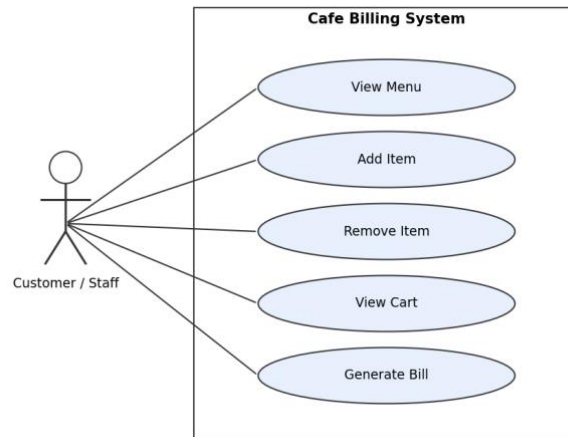


Figure 1: Use case diagram

b. Workflow Diagram

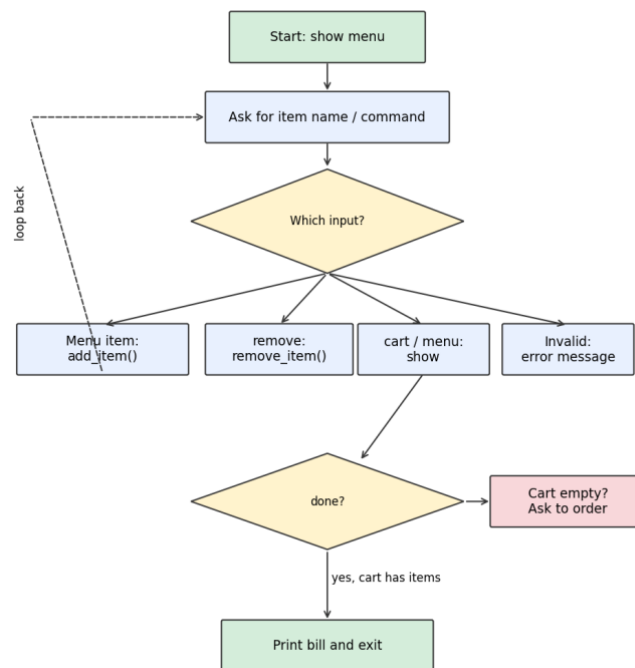


Figure 2: Workflow of the main loop

c. Sequence Diagram

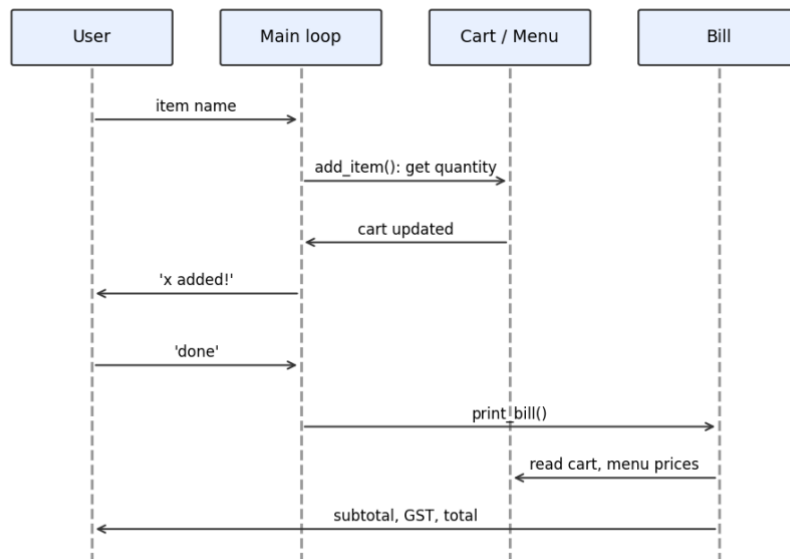


Figure 3: Sequence for adding an item and generating the bill

d.Component Diagram

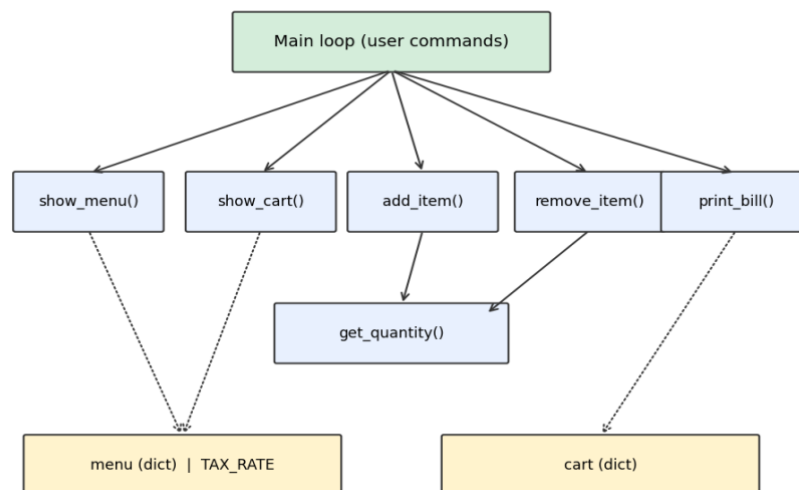


Figure 4: Component / function diagram

e. ER Diagram

Not applicable. The project does not use a database or file storage, only two dictionaries in memory.

7. Design Decisions and Rationale

Decision	Reason
Dictionary for the menu	Looking up a price by item name is easy, and adding a new item is one line.
Dictionary for the cart	Stores item and quantity together, and adding the same item twice just increases the count.

Separate functions	Keeps the main loop short and each task easy to test.
.strip().title() on input	So "pizza", " PIZZA " and "Pizza" all work.
get_quantity() returns 0 on bad input	One function handles all quantity checks for both add and remove.
TAX_RATE constant	If GST changes, only one line needs to be edited.
No file storage	Kept it simple for a first-year project; listed under future enhancements.

8. Implementation Details

The project is written in Python 3 in one file. The main pieces are:

- show_menu() loops over the menu dictionary and prints each item and price in aligned columns.
- get_quantity() reads input, checks it with isdigit(), and makes sure it is at least 1. It returns 0 if the input is not valid.
- add_item(item) gets a quantity and either adds a new entry to the cart or increases the existing one.
- remove_item() shows the cart, asks which item to remove, and reduces the quantity. If it reaches zero or below, the item is deleted from the cart.
- print_bill() loops over the cart, adds up the amounts for the subtotal, calculates GST at 5%, and prints the total.
- The main while True loop reads a command (done, remove, cart, menu or an item name) and uses if/elif/else to decide what to do. break is used to end the loop after printing the bill.

9. Testing Approach

I tested the program manually by running it and trying both normal and wrong inputs.

Test case	Input	Expected result
Add valid item	Pizza, quantity 2	"2 x Pizza added!"
Add same item again	Pizza, quantity 1	Cart shows Pizza x3
Quantity is text	abc	Error message, nothing added
Quantity is zero / negative	0 or -2	"Quantity must be at least 1."
Item not on menu	Sandwich	"Item not found."
Remove item not in cart	Tea (not added)	"That item is not in your cart."
Remove more than in cart	Remove 5 when 2 present	Item removed from cart
Done with empty cart	done	Asks user to order first
Bill calculation	Burger x2	Subtotal 140, GST 7, total 147

10. Challenges Faced

- Handling wrong input for quantity without the program crashing.
- Deciding what to do when the user removes more than they have in the cart.
- Making item names case-insensitive so users don't have to type them exactly.

11. Learnings and Key Takeaways

- How to use dictionaries to store and look up data.
- Breaking a program into functions makes it much easier to debug.
- Always check user input; users will type things you don't expect.
- How while loops, break and if/elif/else work together in a menu-driven program.
- Writing a README and report helped me understand my own code better.

12. Future Enhancements

- Save orders and daily sales to a file or database.
- Add discounts or coupon codes.
- Let the staff add, edit or delete menu items.
- Print bills with date, time and bill number.
- Build a graphical interface or a web version.

13. References

- Python official documentation: <https://docs.python.org/3/>
- Class notes and lab material from the Python course at VIT Bhopal